

Fast Planning for Multi-object Multi-target Throwing

Zhengming Zhu* Yang Liu* Xiao Gao Aude Billard

Abstract—Robot throwing has emerged as a promising technique for improving efficiency in logistics and warehouse automation, by enlarging the workspace and speeding up the process. To significantly increase the throwing system’s throughput, we develop strategies for throwing multiple objects in one swipe. Such multi-object multi-target throwing (MOMT) leverages the large degrees of freedom of anthropomorphic hands. The key is to quickly generate fast and feasible throwing motions, which involves a complex trade-off between short trajectory duration and short planning time. We solve this problem in two stages. Offline, we build a model of the feasible set by combining object’s inverted flying dynamics and the robot’s kinematics and dynamics. Online, we generate feasible throws through fast solution matching and filtering of object’s valid detach state and robot’s feasible state that can compose sequences of throws in less than 5 ms. We validate the framework on a 7-DoF manipulator equipped with a multi-fingered hand. In simulation, coordinated two-object throwing reduces execution time by up to 46% compared to independent single-object planning, and this improvement is maintained when scaling to three objects. Real-world experiments with two objects confirm a 29% reduction; the remaining gap to the theoretical 50% is attributed to inter-throw transition overhead. When target positions are randomly changed mid-execution, the system re-plans and successfully reaches the new targets within 100 ms latency without stopping the robot. These results establish the first unified planning framework for MOMT throwing—demonstrating scalability to multiple objects in simulation and real-world feasibility on two-object tasks—advancing the frontier of high-throughput robotic manipulation. A video summarizing the proposed method and the hardware experiments is available at <https://liuyangdh.github.io/momt-video>.

I. INTRODUCTION

Robot throwing is a challenging dynamic manipulation task, whereby the robot agilely throws an object toward a target. This approach replaces traditional pick and place, with high potential to boost efficiency in many common applications of robots in industry. Yet, past solutions focus exclusively on single-object throwing, where the robot picks and throws one object at a time in repetition.

With the growing availability of low-cost multi-fingered anthropomorphic robot hands, robots now possess the capability of holding multiple objects simultaneously. With multiple objects in hand, it becomes possible to synthesize a single stroke of high-speed motion that throws different objects to their designated targets, thereby drastically increasing object transport efficiency. This is particularly advantageous in high-throughput sorting applications where items must be continuously dispatched to spatially distributed targets, such as parcel routing to multiple destination chutes, recycling

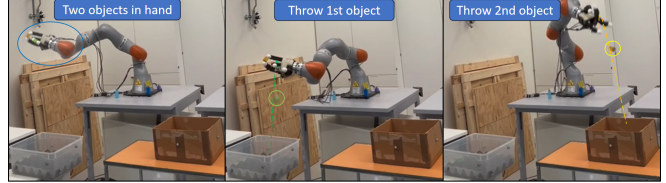


Fig. 1: Multi-object multi-target throwing with a Kuka iiwa 7 robot equipped with an Allegro hand. The robot grasps two objects simultaneously and throws them to different target boxes in a single continuous motion, demonstrating the efficiency of our unified planning framework.

separation into distinct material bins, and warehouse order consolidation. In such settings, a single multi-target throwing stroke amortizes the arm’s transport motion across several items and destinations at once, yielding throughput gains beyond what repeated single-object pick-and-place or single-target throwing can achieve.

However, compared to single-object throwing, *multi-object multi-target* (MOMT) throwing presents a unique challenge: MOMT introduces explosive combinatorial complexity when searching for the best (e.g., time-optimal) throwing motion. This substantially increases planning time, which in turn compromises the throughput of the overall throwing system. Given the demanding and unpredictable MOMT throwing task queries common in industrial settings, it is therefore necessary to strategically balance the time spent on motion planning with the duration of the throwing motion. In this work, we seek a planning framework that provides a favorable trade-off in this dilemma. **List of contributions:**

- We introduce a unified planning framework for multi-object multi-target throwing that combines explicit joint-space release state modeling with offline robot kinematic analysis. By exploiting rotational invariance of fixed-base manipulators, we reduce the dimensionality of the feasibility problem, enabling the generation of diverse feasible throwing configurations in under 5 ms.
- We propose a decomposition approach that separates single-target solution generation from multi-target coordination, and introduce a pruned Greedy search that retains only the top- K candidates per target ranked by joint-space proximity to the current robot state. This enables practitioners to navigate the trade-off between planning time and execution optimality based on application requirements.
- We provide the first real-world demonstration of MOMT throwing, showing reactive multi-target adaptation and a 29% reduction in execution time compared to independent single-object planning, and demonstrating reactive re-planning under dynamic target changes.

*Equal contribution. The authors are with the Learning Algorithms and Systems Laboratory (LASA), EPFL, Switzerland. Corresponding author: Yang Liu (yangliudh@gmail.com).

II. RELATED WORK

A. Robot Throwing Planning

Over the past decades, there has been tremendous progress on robot throwing [1]–[12]. Existing robot throwing motion synthesis methods can be grouped according to how the release state is specified: a) Implicit, b) Explicit Cartesian-space, c) Explicit joint-space.

a) *Implicit release state*: Implicit formulations jointly determine the release state and trajectory. While this unifies optimization and learning-based methods, it typically results in difficult nonconvex optimization problems [13] or sparse-reward MDPs that are hard to train efficiently [14].

b) *Explicit Cartesian release state*: This approach defines the release state in Cartesian space before synthesizing joint motion, leveraging efficient flight dynamics solvers. However, the selected Cartesian state may prove kinematically infeasible, necessitating costly replanning [4] or manual tuning to ensure convergence [12].

c) *Explicit joint-space formulation*: The explicit joint-space formulation specifies the release configuration directly in the robot’s joint state (position and velocity). [7] proposed a planning framework for mobile manipulator throwing, enabling the generation of diverse throwing configurations in joint space in real time. However, [7] assumes that the base is omnidirectional, and hence the horizontal invariance greatly simplifies the planning problem. In this work, we adopt this explicit joint-space formulation and extend it to fixed-base manipulator throwing, and provide efficient planning solutions for multi-object multi-target scenarios. These two challenges were not addressed in previous works.

B. Multi-object manipulation

Holding and manipulating multiple objects simultaneously with a dexterous hand has long been recognized as important for improving robotic throughput. Early work by [15] demonstrated multi-object in-hand manipulation on a 16-DoF robotic hand through human skill transfer, establishing the feasibility of dexterous multi-object control. On the planning side, [16] proposed a grasp synthesis framework that exploits the kinematic redundancy of a multi-fingered hand to generate stable multi-object grasps. More recently, [17] introduced a learning-based pipeline for simultaneously grasping and lifting multiple objects with a dexterous hand, and [18] presented a hand design with reversible fingers that simplifies multi-object grasp planning.

Despite this progress, all existing multi-object manipulation work focuses on quasi-static regimes—grasping, holding, and repositioning. None addresses multi-object *dynamic* manipulation, where objects must be launched at high speed toward distinct targets. In this work, we assume that a suitable multi-object grasp has already been planned (e.g., using [16], [17]) and that the objects are placed correctly in the hand prior to throwing. Under this assumption, our framework addresses the open problem of planning coordinated throwing motions for multiple objects held simultaneously.

III. METHOD

Our framework consists of two components: (1) a **single-target throwing planner** that generates diverse feasible throwing configurations for a fixed-base manipulator in under 5 ms, and (2) a **multi-target coordinator** that coordinates multiple throws to minimize total task time. We describe each in turn.

A. Assumptions

We make the following assumptions throughout this work:

- 1) The manipulator is fixed-base, with the first joint rotating about the vertical axis (common in industrial arms).
- 2) Objects are point masses during flight (no aerodynamic effects or spin).
- 3) Grasp configurations are predefined: each object’s position relative to the end-effector at release is known.
- 4) The workspace is obstacle-free during the throwing motion.
- 5) Object release is instantaneous, while in-hand sliding/pivoting are neglected.

B. Single-target Throwing Planner

1) *Key Insight*: The single-target throwing planner builds upon the spatially-discretized dictionary, *Velocity Hedgehog*, proposed in [7] for mobile manipulator throwing. Counterintuitively, fixed-base throwing is *more challenging* than mobile-base throwing: the mobile base provides additional degrees of freedom that actually *simplify* the planning problem. Specifically, [7] assumes an omnidirectional mobile base, where the base position can be determined *after* the arm motion is planned—the horizontal invariance decouples the problem.

For fixed-base manipulators, this decoupling is lost. A straightforward extension would require discretizing the end-effector position (x, y, z) along with throwing direction (ϕ, γ) , resulting in a 5-dimensional mesh that is computationally intractable.

We exploit the fact that for common manipulators where the first joint rotates around the vertical axis, the throwing problem exhibits *rotational invariance*. This allows us to:

- Replace 2D horizontal position (x, y) with 1D radial distance d , reducing to a 4D mesh (z, d, ϕ, γ) .
- Compute yaw angle ϕ *analytically* from the target position, achieving zero discretization error in landing position.

The result is a compact dictionary with effectively 4 discretized dimensions, which we call the **maximum velocity dictionary**. Combined with the **Backward Reachable Tube (BRT)** encoding valid release states from flight dynamics, the single-target throwing planner generates feasible throwing configurations in under 5 ms.

It is worth noting that this reduction relies on Assumption 1; for general manipulator mountings whose first joint is not vertical, rotational invariance no longer holds and one must discretize the full end-effector position (x, y, z) , enlarging the offline dictionary, while the online planning procedure remains unchanged.

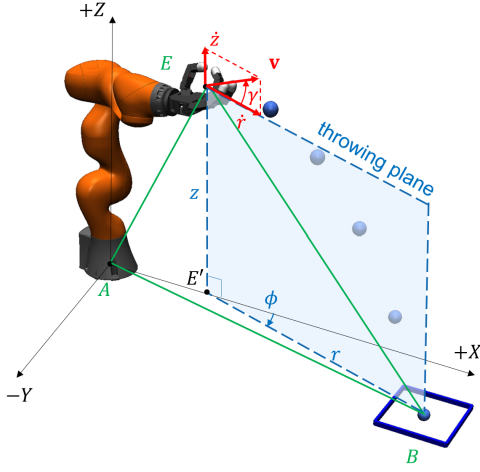


Fig. 2: Geometric modeling at release.

2) *Geometric Modeling*: We define the throwing geometry using triangle $\triangle AEB$ in 3D space (Fig. 2, Table I).

Symbol	Description
$(q, \dot{q}) \in \mathbb{R}^{2n}$	Joint release state for n-DoF robot
$A p^B \in \mathbb{R}^3$	Target position in robot frame
$A p^E \in \mathbb{R}^3$	Object position at release in robot frame
$\mathbf{v} \in \mathbb{R}^3$	Object velocity at release
d_E	Horizontal distance from E to robot base A
d_B	Horizontal distance from B to robot base A
r	Horizontal distance from E to B
z	Height of object relative to target
ϕ	Yaw angle (throwing direction)
γ	Pitch angle (throwing elevation)
ψ_E, ψ_B	Azimuth of end-effector and target

TABLE I: Notation for geometric modeling

The throwing triangle parameters are computed via trigonometry (Fig. 2):

$$r^2 = d^2 + d_B^2 - 2dd_B \cos(\psi_B - \psi_E) \quad (1)$$

$$\phi = \arctan\left(\frac{d_B \sin(\psi_B - \psi_E)}{d_B \cos(\psi_B - \psi_E) - d}\right) \quad (2)$$

Given target position B relative to the robot base A , we seek a release state $(r, z, \dot{r}, \dot{z})$ satisfying joint limits such that the object lands on the target under gravity.

Thus, the throwing problem is formulated as:

$$\text{Find } \{r, z, \dot{r}, \dot{z}, q, \dot{q}\} \quad (3)$$

subject to:

$$\mathbf{v} = J(q)\dot{q} \quad (4)$$

$$\frac{\mathbf{v}_{xy}}{\|\mathbf{v}_{xy}\|} = \frac{{}^E p_{xy}^B}{\|{}^E p_{xy}^B\|} \quad (5)$$

$$(r(q), z(q), \|\mathbf{v}_{xy}\|, \mathbf{v}_z) \in \text{BRT}_{f_{\text{fly}}, \mathcal{X}} \quad (6)$$

$$q_{\min} \leq q \leq q_{\max}, \quad \dot{q}_{\min} \leq \dot{q} \leq \dot{q}_{\max} \quad (7)$$

where r and z depend on q implicitly via forward kinematics and the throwing geometry (Fig. 2); (5) enforces that the horizontal release velocity points toward the target, constraining the throw to lie in plane $EE'B$; and $\text{BRT}_{f_{\text{fly}}, \mathcal{X}}$ is the set of valid release states in that plane, which we explain next.

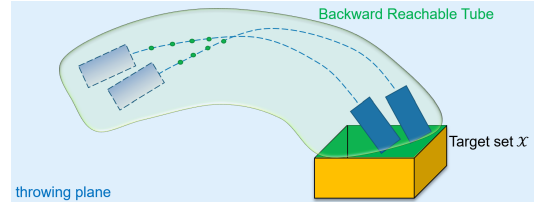


Fig. 3: The Backward Reachable Tube (BRT) is the set of release states whose trajectories reach the landing target set $\mathcal{X}$.

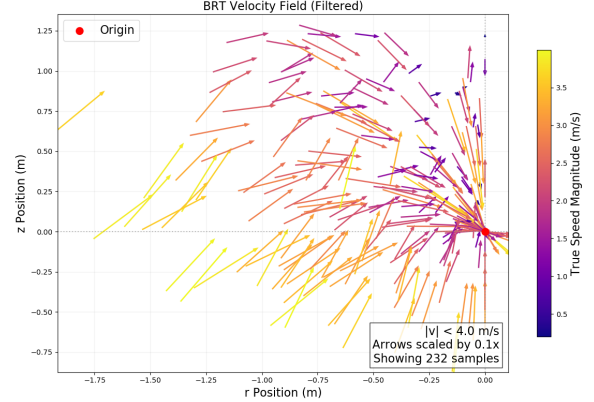


Fig. 4: Visualization of the BRT. At each spatial position, arrows show valid release velocities.

3) *Backward Reachable Tube (BRT)*: The BRT encodes all release states from which an object can reach the target. We define the flight state as $\xi = [r, z, \dot{r}, \dot{z}]^T \in \mathbb{R}^4$, governed by: $\dot{\xi} = f_{\text{fly}}(\xi)$, where f_{fly} describes the flight dynamics (e.g., projectile motion under gravity). Let $\zeta_{f_{\text{fly}}, \xi^0}(t)$ denote the trajectory starting from state ξ^0 . Given a landing target set $\mathcal{X} \subseteq \mathbb{R}^4$ specifying allowable landing positions and velocities, the BRT is:

$$\text{BRT}_{f_{\text{fly}}, \mathcal{X}} = \{\xi^0 \mid \exists t \geq 0, \zeta_{f_{\text{fly}}, \xi^0}(t) \in \mathcal{X}\} \quad (8)$$

In words: the BRT contains all states from which the object eventually lands in $\mathcal{X}$ (Fig. 3).

Building the BRT: We sample states in $\mathcal{X}$ and integrate the ODE *backwards* in time to trace trajectories to their release states. For projectile motion (gravity only), the flight time and landing position have closed-form expressions:

$$\Delta t_{\text{fly}} = \frac{\dot{z}_d + \sqrt{\dot{z}_d^2 + 2g(z_d - z_{\text{land}})}}{g}, \quad r_{\text{land}} = r_d + \dot{r}_d \Delta t_{\text{fly}} \quad (9)$$

where $(r_d, z_d, \dot{r}_d, \dot{z}_d)$ is the release state. For complex dynamics (e.g., with air drag [19]), numerical integration can be used.

The resulting BRT is a 4D volume in state space: at each position (r, z) , there exists a *set* of valid release velocities $(\dot{r}, \dot{z})$ that will land the object in the target (Fig. 4). This multiplicity of solutions is key to our approach—it provides flexibility for matching with robot kinematic constraints.

4) *Maximum Velocity Dictionary*: The maximum velocity dictionary stores the robot's maximum achievable end-effector velocity for each throwing configuration, organized as a 4D mesh over (z, d, ϕ, γ) .

Algorithm 1 Maximum Velocity Dictionary (Offline)

```

1: Input: Robot model, joint position and velocity limits
   ( $q_{\min}, q_{\max}, \dot{q}_{\min}, \dot{q}_{\max}$ )
2: Output:  $v_{\max}(z, d, \phi, \gamma)$ ,  $q(z, d, \phi, \gamma)$ 
3: Sample joint configurations  $Q$  on a grid
4: Filter configurations with small Jacobian singular values

5: Group by height  $z$  and distance  $d$ 
6: for each  $(z, d, \phi, \gamma)$  bin do
7:   for each configuration  $q$  in bin do
8:      $v \leftarrow$  solve LP for max velocity along  $(\phi, \gamma)$ 
9:     if  $v > v_{\max}(z, d, \phi, \gamma)$  then
10:       $v_{\max}(z, d, \phi, \gamma) \leftarrow v$ ;  $q(z, d, \phi, \gamma) \leftarrow q$ 
11:     end if
12:   end for
13: end for

```

Since the first joint q_0 rotates around the vertical axis, we exclude it during offline construction (setting $q_0 = 0$). During online matching, q_0 is computed analytically to align the throwing triangle with the target azimuth:

$$q_0 = \psi_B - \arccos\left(\frac{d^2 + d_B^2 - r^2}{2dd_B}\right) \quad (10)$$

This yields zero discretization error in the horizontal landing position. Therefore, for each bin, we store (Algorithm 1):

- Maximum feasible velocity $v_{\max}(z, d, \phi, \gamma)$
- Corresponding joint configuration q

The maximum velocity at configuration q along direction (ϕ, γ) is found by solving:

$$\max_s \quad s \quad \text{s.t.} \quad \dot{q}_{\min} \leq J^\dagger(q)\vec{v} \leq \dot{q}_{\max} \quad (11)$$

where $\vec{v} = s[\cos \gamma \cos \phi, \cos \gamma \sin \phi, \sin \gamma]^\top$ and $J^\dagger(q)$ is the Jacobian pseudoinverse. This is a linear program (LP) solved efficiently as a subroutine.

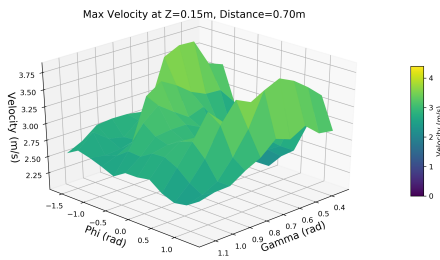


Fig. 5: Visualization of the maximum velocity dictionary for the Kuka iiwa 7. The surface shows the maximum achievable end-effector velocity as a function of throwing direction (ϕ, γ) at a fixed height and distance.

Fig. 5 visualizes the maximum velocity dictionary for the Kuka iiwa 7. The distribution reveals kinematic limits across configurations: for this robot, the maximum Cartesian velocity is ~ 3.75 m/s, and the maximum throwing distance (determined by flight dynamics) is ~ 1.5 m.

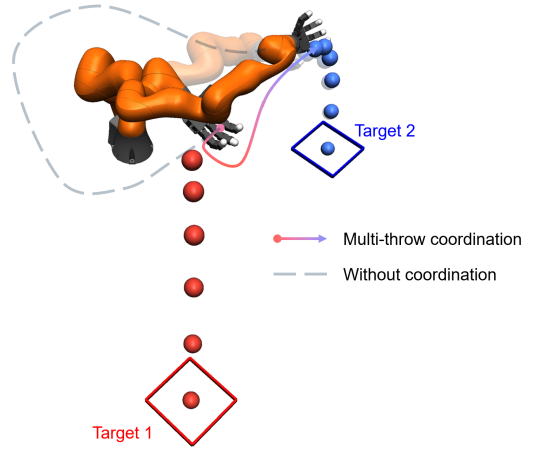


Fig. 6: Motivation for multi-throw coordination. The single-target planner yields multiple valid configurations per target (red for Target 1, blue for Target 2). Choosing configurations independently may result in large joint-space transitions (dashed arc), while coordinated selection minimizes total execution time (solid arrow).

5) Solution Matching: Given a target position, we find feasible throwing configurations by matching the BRT (valid release states) with the maximum velocity dictionary (achievable robot states).

A release state $\xi = (r, z, \dot{r}, \dot{z})$ from the BRT is feasible if the required velocity $v = \|(\dot{r}, \dot{z})\|$ does not exceed the robot's capability:

$$v \leq v_{\max}(z, d, \phi, \gamma), \quad \text{where } \gamma = \arctan(\dot{r}/\dot{z}) \quad (12)$$

Efficient matching: For each height z and pitch γ , we have two sorted sets: BRT states by required velocity and maximum velocity dictionary entries by maximum velocity. Matching reduces to a linear sweep with complexity $\mathcal{O}(N_{\text{BRT}} + N_{\text{dict}})$ instead of naive $\mathcal{O}(N_{\text{BRT}} \cdot N_{\text{dict}})$.

After matching, we filter candidates based on task-specific criteria (e.g., prefer configurations with safety margins) and output the final throwing configuration $(q, \dot{q})$.

6) Trajectory Generation: Given the current robot state and a candidate throwing configuration from Solution Matching, we use Ruckig [20] to generate time-optimal jerk-limited trajectories. Ruckig provides both the executable trajectory and its duration in closed form (< 0.5 ms for 7-DoF). This trajectory generation is also used in the multi-target coordinator to evaluate and compare total execution time across multi-throw sequences.

C. Multi-target Coordinator

The single-target throwing planner outputs a *set* of valid throwing configurations per target. When multiple targets must be served, the choice of which configuration to use for each target is not independent: two configurations that are individually valid may be far apart in joint space (Fig. 6, dashed arc), leading to long transition trajectories. The multi-target coordinator selects one configuration per target *jointly*, minimizing total execution time (solid arrow).

For multi-object multi-target (MOMT) throwing, we must coordinate multiple throws to minimize total task time. This

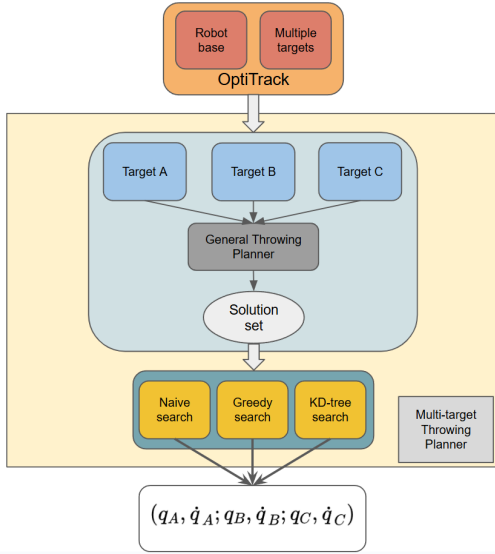


Fig. 7: Multi-target coordinator: Given multiple targets, the single-target throwing planner generates candidate solutions for each. The coordinator searches for the best combination minimizing total execution time.

introduces a combinatorial challenge: with N objects and M candidate configurations per object, there are M^N possible combinations. We consider three search strategies (Fig. 7) trading off computation time vs. execution time:

1) *Naive Search*: Solve each target independently using single-target throwing planner, selecting the locally optimal configuration for each. This ignores inter-throw coordination, leading to suboptimal total execution time (e.g., large joint motions between consecutive throws).

Algorithm 2 Naive Search (Online)

```

1: Input: Targets  $\mathcal{T} = \{t_1, \dots, t_N\}$ , robot base  $A$ 
2: Output: Trajectory sequence
3: for each target  $t_i \in \mathcal{T}$  do
4:    $(q_i^*, \dot{q}_i^*) \leftarrow \text{SingleTargetPlan}(t_i, A)$  // Local optimum
5: end for
6: return TrajectoryGenerator( $\{(q_i^*, \dot{q}_i^*)\}$ )

```

2) *Greedy Search*: Generate multiple candidate configurations per target, then search over all combinations. This finds globally better solutions but has exponential complexity $\mathcal{O}(M^N)$.

3) *Pruned Greedy Search*: To control computational cost, we introduce a pruned variant of Greedy search with parameter K : for each target, only the top- K candidates ranked by the heuristic distance $\|q_c - q_0\|_2 + 2\|\dot{q}_c - \dot{q}_0\|_2$, where $(q_0, \dot{q}_0)$ is the current robot state and $(q_c, \dot{q}_c)$ is a candidate configuration. The intuition is that configurations close to the current robot state in joint position and velocity require shorter transition trajectories, and are therefore more likely to yield low total execution time.

IV. EXPERIMENT

In this section, we evaluate the proposed MOMT throwing framework through simulation studies and real-world experiments. Our evaluation investigates three key questions:

Algorithm 3 Greedy Search (Online)

```

1: Input: Targets  $\mathcal{T}$ , robot base  $A$ 
2: Output: Time-optimal trajectory sequence
3: for each target  $t_i$  do
4:    $\mathcal{C}_i \leftarrow \text{GetCandidates}(t_i, A)$  // Multiple candidates
5: end for
6:  $T_{\min} \leftarrow \infty$ 
7: for each combination  $(c_1, \dots, c_N) \in \mathcal{C}_1 \times \dots \times \mathcal{C}_N$  do
8:    $T \leftarrow$  total trajectory duration for this combination
9:   if  $T < T_{\min}$  then
10:     $T_{\min} \leftarrow T$ ; save best combination
11:   end if
12: end for
13: return best trajectory sequence

```

- 1) **Planning-execution trade-off**: How does the pruning parameter K in Greedy search affect the balance between computation time and execution optimality?
- 2) **Scalability**: How does the framework perform as the number of objects increases?
- 3) **Real-world validity**: Is the planning framework effective on physical hardware, and can the computational efficiency enable reactive re-planning?

We first systematically study the planning-execution trade-off in simulation where extensive trials can be conducted efficiently (Sections IV-A–IV-B), then validate the framework on real hardware with reactive throwing demonstrations (Section IV-C).

A. Simulation Study: Two-Object Throwing

We characterize the planning-execution trade-off in simulation using a Kuka iiwa 7 model with the same kinematic parameters and joint limits as the real robot. All timing measurements were performed on Apple M2 Pro (3.49GHz, 32GB).

Beyond computation and execution time, we also report energy cost, measured from the initial holding configuration to release of the last object (excluding the homing phase). Joint torques follow from the planned trajectory via rigid-body inverse dynamics:

$$\tau(t) = \mathbf{M}(\mathbf{q}_{\text{des}})\ddot{\mathbf{q}}_{\text{des}} + \mathbf{C}(\mathbf{q}_{\text{des}}, \dot{\mathbf{q}}_{\text{des}})\dot{\mathbf{q}}_{\text{des}} + \mathbf{g}(\mathbf{q}_{\text{des}}) \quad (13)$$

where $\mathbf{M}$, $\mathbf{C}$, and $\mathbf{g}$ are the mass, Coriolis, and gravity terms. Since most manipulators dissipate braking energy as heat rather than recovering it, we count only positive mechanical work:

$$E = \int_{t_0}^{t_{\text{release}}} \max(\tau(t) \cdot \dot{\mathbf{q}}_{\text{des}}(t), 0) dt \quad (14)$$

where t_0 is the start of the throwing motion and t_{release} is the time at which the last object is released.

1) *Trade-off Analysis*: Table II presents the trade-off between computation time and execution time for varying pruning levels K in the two-object scenario ($n = 10$ target configurations). The results reveal:

TABLE II: Planning-execution trade-off for two objects (simulation)

Method	Computation (s)	Execution (s)	Energy (J)
Naive ($K = 1$)	0.06 ± 0.01	6.38 ± 1.69	87.46 ± 58.74
Greedy- $K5$	0.06 ± 0.01	5.10 ± 1.08	62.56 ± 40.52
Greedy- $K10$	0.06 ± 0.01	4.40 ± 0.70	53.56 ± 36.68
Greedy- $K20$	0.07 ± 0.01	4.21 ± 0.68	43.50 ± 24.63
Greedy- $K50$	0.12 ± 0.01	3.61 ± 0.41	42.39 ± 20.37
Greedy- $K80$	0.19 ± 0.01	3.51 ± 0.25	47.38 ± 21.90
Greedy- $K100$	0.27 ± 0.01	3.47 ± 0.23	47.92 ± 14.12
Greedy (full)	0.55 ± 0.11	3.45 ± 0.25	46.27 ± 19.90

- **Diminishing returns:** Increasing K beyond 50 yields marginal execution improvement while computation time continues to grow.
- **Sweet spot:** At $K = 80$, the pruned Greedy achieves 98.3 % of the full Greedy’s execution performance while requiring only 0.19 s of computation.

B. Scalability and Planning-Execution Trade-off for More Objects

To evaluate the scalability of our framework, we extended the simulation to scenarios with three and four objects ($n = 10$ target configurations each). Table III and Table IV present the planning-execution trade-offs.

TABLE III: Planning-execution trade-off for three objects (simulation)

Method	Computation (s)	Execution (s)	Energy (J)
Naive ($K = 1$)	0.11 ± 0.01	9.57 ± 1.58	128.74 ± 74.57
Greedy- $K5$	0.10 ± 0.01	6.98 ± 0.84	73.08 ± 34.82
Greedy- $K10$	0.12 ± 0.01	6.41 ± 0.56	73.42 ± 23.51
Greedy- $K20$	0.29 ± 0.01	5.85 ± 0.33	62.78 ± 19.99
Greedy- $K50$	2.81 ± 0.06	5.45 ± 0.26	67.35 ± 18.84
Greedy- $K80$	11.11 ± 0.22	5.27 ± 0.21	76.26 ± 21.03
Greedy- $K100$	21.81 ± 0.38	5.22 ± 0.17	67.86 ± 27.24
Greedy (full)	68.13 ± 13.85	5.18 ± 0.22	78.41 ± 30.56

TABLE IV: Planning-execution trade-off for four objects (simulation)

Method	Computation (s)	Execution (s)	Energy (J)
Naive ($K = 1$)	0.15 ± 0.01	13.59 ± 1.96	195.11 ± 66.86
Greedy- $K5$	0.15 ± 0.01	9.54 ± 1.05	95.19 ± 41.76
Greedy- $K10$	0.40 ± 0.01	8.95 ± 0.82	94.79 ± 48.02
Greedy- $K20$	3.90 ± 0.08	8.16 ± 0.41	100.74 ± 21.65
Greedy- $K30$	18.50 ± 0.22	7.73 ± 0.25	101.50 ± 27.27
Greedy- $K50$	138.45 ± 1.35	7.48 ± 0.20	87.10 ± 33.74

Key observations from the multi-object experiments ($N = 3, 4$) include:

- **Scalability and Pruning Necessity:** The computational cost grows exponentially with object count ($O(K^N)$), rendering exhaustive search intractable (e.g., 68.1 s for $N = 3$, > 138 s for $N = 4$ with $K = 50$). However, the pruned **Greedy- $K20$** strategy effectively mitigates this, maintaining tractable computation (0.29 s for $N = 3$, 3.9 s for $N = 4$) while capturing over **85%** of the execution time improvement compared to the Naive baseline.
- **Consistent Coordination Benefits:** The framework demonstrates robust performance across complexities. The execution time reduction from Naive to the best Greedy solution remains stable at approximately **45%**

for both 3-object (45.8%) and 4-object (44.9%) scenarios, indicating that temporal coordination benefits do not diminish with increased task size.

- **Energy Efficiency:** Coordination significantly optimizes energy usage alongside time. Particularly in the 4-object case, the Naive approach consumes nearly double the energy (195.11 J) compared to Greedy variants (≈ 95 –100 J), as the optimized sequence minimizes unnecessary holding torques and inefficient transitions.

C. Real-World Validation

Having characterized the planning-execution trade-off in simulation, we then deployed and validated the framework on our available lab hardware using a two-object throwing task, confirming its real-world viability.

1) *Experimental Setup:* The real-world experiments were conducted using a Kuka iiwa 7 robot equipped with an Allegro hand. An OptiTrack motion capture system provided ground-truth measurements of object flight trajectories, target positions, and the robot base frame. The targets were boxes of 0.3×0.3 m. The planned throwing trajectories are tracked by a joint impedance controller.

The thrown objects were wooden blocks (mass: 15 g, dimensions: $3 \times 3 \times 3$ cm). For multi-object grasping, we identified a fingertip configuration on the Allegro hand that can reliably hold two objects while minimizing release interference between them (Fig. 8). Finding such configurations required manual tuning, as the Allegro hand’s kinematics impose constraints on feasible multi-object grasps.



Fig. 8: Left: Two objects held by fingertips in the Allegro hand using a manually-tuned configuration that minimizes release interference. Right: Release delay measurement using an ATI nano F/T sensor.

To account for the release delay inherent in finger opening, we command the hand to open before arriving at the nominal throwing state. The release delay was measured via an ATI nano F/T sensor (Fig. 8) and found to be approximately 40–50 ms. This delay is compensated in the release timing.

TABLE V: Real-world validation (two objects, $n = 10$ configurations)

Method	Computation	Execution	Throws Landed
Naive	0.06 ± 0.02 s	5.20 ± 0.98 s	14/20
Greedy	69.8 ± 32.3 s	3.69 ± 0.37 s	14/20

We first confirm that the planning-execution trade-off observed in simulation transfers to real hardware. Table V summarizes results across 10 target configurations. Consistent with simulation, Greedy search reduces execution time by 29% compared to Naive search, at the cost of significantly

longer planning time. The supplementary video shows both strategies side-by-side.

Notably, both strategies land the same fraction of throws (14/20; Table V). This is expected rather than incidental: the coordination strategy governs *which* release states are selected and in what order—determining execution time and energy—but not the accuracy with which a chosen throw is realized. Landing accuracy instead depends on release dynamics, which are identical across both methods. The observed misses stem primarily from our simplified release model (Assumption 5): instantaneous release with neglected in-hand sliding and pivoting, compounded by release-timing variance and tracking error near the velocity limits. Improving landing accuracy is therefore orthogonal to our contribution and could be addressed by release-aware modeling and control—robustifying throws against release uncertainty for center-of-mass grasps [9], modeling the transient release dynamics of eccentric grasps [21], or refining throws from past experience [22]. These act at the level of individual throws and integrate directly with our planner.

2) *Reactive Multi-Target Throwing*: The key advantage of fast online planning is enabling reactive behavior. We demonstrate this by randomly changing target positions during execution.

Figure 9 illustrates the reactive throwing experiment. When a target position change is detected, the system:

- 1) Queries the single-target planner for new feasible configurations for the updated target.
- 2) Updates the multi-target coordination solution if the robot has not yet committed to the current throw.
- 3) Generates a new trajectory and continues execution.

The total re-planning latency (single-target planning + multi-target coordination + trajectory generation) remains under 100 ms with Naive search, enabling smooth adaptation without stopping the robot. This reactive capability is only possible because of the computational efficiency of our framework—full Greedy search would require stopping for over a minute to re-plan.

D. Discussion: Why Not Precompute Everything?

A natural question is whether the entire multi-target coordination problem can be solved offline. For the two-object case, we discretize the target workspace (0.05 m resolution in xy , 0.04 m in z , annular region $1.25 \leq \|(x, y)\| \leq 1.4$ m) and solve the full Greedy search for all valid target pairs, yielding ~ 6.4 M precomputed configuration pairs (6D key: two target positions; 28D value: joint angles and velocities for both throws). At runtime, a KD-tree nearest-neighbor query retrieves the closest precomputed solution. This achieves the fastest online planning (< 5 ms, dominated by trajectory generation), but has three limitations: (1) it requires hours of offline computation; (2) it introduces discretization error, as the retrieved solution corresponds to the nearest grid point rather than the exact query target; and (3) it does not scale beyond two objects—for three objects, the precomputation space grows to ~ 16 billion combinations, making brute-force offline solving infeasible.

TABLE VI: Search strategy comparison

Method	Planning time	Execution quality	Arbitrary targets?	Scales to N objects?
Naive	< 100 ms	Baseline	Yes	Yes
Greedy- K	0.1–20 s	Improved	Yes	Yes
Greedy (full)	> 60 s	Best	Yes	Yes
Precomputed	< 5 ms	Best*	No	No [†]

*Execution quality matches full Greedy for nearby grid points but is subject to discretization error bounded by the grid resolution (~ 0.025 m).

[†]Currently implemented for 2 objects only; see text.

Table VI summarizes the trade-offs across all strategies. Note that the planning times reported there refer to multi-target coordination; single-target solution generation adds a fixed cost of under 5 ms per target, independent of the coordination strategy. Real-time operation is therefore a property of the strategy rather than the planner as a whole: single-target generation is always real-time, and only Naive or precomputed coordination preserves real-time end-to-end performance for reactive use, whereas the Greedy variants target offline or latency-tolerant planning where execution optimality is paramount.

V. DISCUSSION AND FUTURE WORK

A. Synergy between grasping and throwing planning

Scaling to three or more objects with the current hand proved impractical due to workspace limitations and inter-object interference during release.

We note that this grasping limitation is not fundamental to our planning framework. Recent advances in dexterous hand design [18] offer improved multi-object manipulation capabilities. Additionally, specialized gripper mechanisms with object storage in the palm [23] provide an alternative approach where multiple objects can be stored and released sequentially. Our MOMT planning framework readily extends to such hardware, as it only requires knowledge of the object positions relative to the end-effector at release time. The planning of grasp configurations to minimize inter-object interference during release remains an interesting direction for future work.

B. Collision-Free Motion Planning

The current framework assumes an obstacle-free workspace. Extending it to cluttered environments requires integrating collision constraints into trajectory optimization while preserving the computational efficiency that enables reactive re-planning.

C. Flight Dynamics

We model objects as point masses in free flight (Assumption 2), which holds well for the compact, dense blocks used in our experiments but degrades for light or aerodynamically significant objects. Since the BRT is derived directly from the flight model, relaxing this assumption requires only recomputing the BRT by numerically integrating the true dynamics, including drag and spin [19], while the online matching and multi-target coordination remain unchanged.



Fig. 9: Reactive multi-object multi-target throwing. When target positions change mid-execution, the system re-plans and adapts in real-time. See the supplementary video for the full demonstration.

VI. CONCLUSIONS

We presented a unified planning framework for multi-object multi-target (MOMT) dexterous throwing with fixed-base manipulators. Exploiting the rotational invariance of common manipulator kinematics, we reduced the feasibility problem to a compact 4D representation that supports online solution matching in under 5 ms per target, and introduced a pruned Greedy search that lets practitioners trade planning time against execution optimality.

On a Kuka iiwa 7 with an Allegro hand, the framework reduced execution time by 29% over independent single-object planning and enabled reactive re-planning under dynamic target changes within 100 ms. To our knowledge, this is the first unified planning framework for MOMT throwing, validated in both simulation and real-world two-object experiments. Its modular decomposition into single-target planning and multi-target coordination provides a foundation for future capabilities such as dexterous grasp planning and collision-free trajectory optimization.

REFERENCES

- [1] E. W. Aboaf, C. G. Atkeson, and D. J. Reinkensmeyer, "Task-level robot learning: Ball throwing," 1987.
- [2] K. M. Lynch and M. T. Mason, "Dynamic nonprehensile manipulation: Controllability, planning, and experiments," *The International Journal of Robotics Research*, vol. 18, no. 1, pp. 64–92, 1999.
- [3] T. Senoo, A. Namiki, and M. Ishikawa, "High-speed throwing motion based on kinetic chain approach," in *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2008, pp. 3206–3211.
- [4] Y. Zhang, J. Luo, and K. Hauser, "Sampling-based motion planning with dynamic intermediate state objectives: Application to throwing," in *2012 IEEE International Conference on Robotics and Automation*. IEEE, 2012, pp. 2551–2556.
- [5] A. Pekarovskiy and M. Buss, "Optimal control goal manifolds for planar nonprehensile throwing," in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2013, pp. 4518–4524.
- [6] A. Zeng, S. Song, J. Lee, A. Rodriguez, and T. Funkhouser, "Tossing-bot: Learning to throw arbitrary objects with residual physics," *IEEE Transactions on Robotics*, vol. 36, no. 4, pp. 1307–1319, 2020.
- [7] Y. Liu, A. Nayak, and A. Billard, "A solution to adaptive mobile manipulator throwing," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 1625–1632.
- [8] H. Kasaei and M. Kasaei, "Throwing objects into a moving basket while avoiding obstacles," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 3051–3057.
- [9] Y. Liu and A. Billard, "Tube acceleration: robust dexterous throwing against release uncertainty," *IEEE Transactions on Robotics*, 2024.
- [10] A. Zermane, N. Dehio, and A. Kheddar, "Planning impact-driven logistic tasks," *IEEE Robotics and Automation Letters*, 2024.
- [11] L. Werner, F. Nan, P. Eyschen, F. A. Spinelli, H. Yang, and M. Hutter, "Dynamic throwing with robotic material handling machines," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2024, pp. 98–104.
- [12] Y. Ma, Y. Liu, K. Qu, and M. Hutter, "Learning accurate whole-body throwing with high-frequency residual policy and pullback tube acceleration," *arXiv preprint arXiv:2506.16986*, 2025.
- [13] F. Lombai and G. Szederkényi, "Throwing motion generation using nonlinear optimization on a 6-degree-of-freedom robot manipulator," in *2009 IEEE International Conference on Mechatronics*. IEEE, 2009, pp. 1–6.
- [14] H. Munn, B. Tidd, P. Böhm, M. Gallagher, and D. Howard, "Whole-body dynamic throwing with legged manipulators," *arXiv preprint arXiv:2410.05681*, 2024.
- [15] E. Oztop, L.-h. Lin, M. Kawato, and G. Cheng, "Dexterous skills transfer by extending human body schema to a robotic hand," in *2006 6th IEEE-RAS international conference on humanoid robots*. IEEE, 2006, pp. 82–87.
- [16] K. Yao and A. Billard, "Exploiting kinematic redundancy for robotic grasping of multiple objects," *IEEE Transactions on Robotics*, vol. 39, no. 3, pp. 1982–2002, 2023.
- [17] Y. Li, B. Liu, Y. Geng, P. Li, Y. Yang, Y. Zhu, T. Liu, and S. Huang, "Grasp multiple objects with one hand," *IEEE Robotics and Automation Letters*, vol. 9, no. 5, pp. 4027–4034, 2024.
- [18] X. Gao, K. Yao, K. Junge, J. Hughes, and A. Billard, "A detachable crawling robotic hand," *Nature Communications*, vol. 17, no. 1, p. 428, 2026.
- [19] M. V. Lubarda and V. A. Lubarda, "A review of the analysis of wind-influenced projectile motion in the presence of linear and nonlinear drag force," *Archive of Applied Mechanics*, vol. 92, no. 7, pp. 1997–2022, 2022.
- [20] L. Berscheid and T. Kröger, "Jerk-limited real-time trajectory generation with arbitrary target states," *Robotics: Science and Systems XVII*, 2021.
- [21] Y. Liu and A. Billard, "On transient release dynamics in robot throwing: A sliding pivot model," *IEEE Transactions on Robotics*, vol. 42, pp. 132–151, 2025.
- [22] Y. Liu, B. Da Costa, and A. Billard, "Learning to throw-flip," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 3209–3216.
- [23] J. Eom, S. Y. Yu, W. Kim, C. Park, K. Y. Lee, and K.-J. Cho, "Mogrip: Gripper for multiobject grasping in pick-and-place tasks using translational movements of fingers," *Science Robotics*, vol. 9, no. 97, p. eado3939, 2024.